

Technical Assignment: Document Lifecycle, Template & PDF Rendering Engine

Critical Evaluation Requirement: Downloadable PDF

Payload + Template = Downloadable PDF.

The examiner must be able to download and inspect the compiled PDF directly. Your service must expose a dedicated endpoint (e.g., GET /api/documents/:id/pdf) that merges the document payload data with your registered template and directly streams or serves a downloadable .pdf file (application/pdf).

Overview

This assessment evaluates your ability to build a backend service that models document lifecycles, maintains immutable audit trails, registers structured document templates, and renders dynamic payloads into downloadable PDF documents.

Tech Stack

- **Runtime:** Bun
- **Framework:** Nitro (or H3)
- **Language:** TypeScript
- **Database:** PostgreSQL or SQLite
- **Validation:** Zod
- **Containerization:** Docker / Docker Compose

Core Requirements

1. Document State Machine

Each document record must exist in exactly one lifecycle state:

- DRAFT (Initial state)
- SENT
- SIGNED (Terminal state)
- VOIDED (Terminal state)

Transition Rules:

- DRAFT -> SENT or VOIDED
- SENT -> SIGNED or VOIDED
- SIGNED -> No further transitions permitted.
- VOIDED -> No further transitions permitted.
- Any illegal state transition must return a 400 Bad Request with an explanatory error message.

2. Immutable Audit Trail

Every state transition must persist an immutable audit log capturing:

- Document ID
- Previous state
- Next state
- Actor identifier (e.g., email or user ID)
- ISO 8601 timestamp
- Optional note or transition reason

3. Document Template Implementation

Implement a document template that reproduces the visual hierarchy, layout, and styling of the provided design asset:

- **Reference Target:** Implement the **VIP Weekend Pass / Event Ticket** design shown above.



- **Dynamic Data Schema:** Structure a validation schema capturing all dynamic fields (series, ticket number, event titles, dates, admission rules, venue details, and gate information etc).
- **Sample / Placeholder Data:** Provide default placeholder fixtures reflecting the fields in the sample pass.
- **Rendering Component:** Construct the visual template matching the styling and structural zones of the mock (main pass details, tear-off stub, reference badge, QR container).
- **Registration:** Register the template within your system so it can be dynamically compiled with runtime payloads.

4. PDF Compilation & Download Pipeline

- When requested, the service must inject the document's dynamic data payload into the template and compile it into a valid PDF binary.
- The compiled PDF must be returned with proper headers (Content-Type: application/pdf, Content-Disposition: attachment; filename="...") for direct browser/client download.

API Specification

POST /api/documents

Creates a new document in DRAFT state using a registered template.

- **Payload:**

```
JSON
{
  "templateId" "jazz-festival-ticket"
  "title" "VIP Weekend Pass - Jazz Festival 2026"
  "createdBy" "organizer@riversidejazz.fest"
  "data"
  "series" "017"
  "ticketNumber" "RJF26-00482"
  "venue" "Willow Bend Riverfront Park"
}
```

- **Response:** 201 Created

GET /api/documents/:id

Fetches current document metadata, stored payload, and lifecycle status.

- **Response:** 200 OK

POST /api/documents/:id/transition

Triggers a lifecycle state transition.

- **Payload:**

```
JSON
{
  "transition" "cancel"
}
```

```
"action" "SENT"
```

```
"actor" "boxoffice@riversidejazz.fest"
```

```
"note" "Issued ticket to attendee"
```

- **Response:** 200 OK on success, or 400 Bad Request if invalid.

GET /api/documents/:id/audit-trail

Returns the complete chronological audit trail for the document.

- **Response:** 200 OK

GET /api/documents/:id/pdf

Compiles the document's payload with its template and serves the file.

- **Response:** Binary PDF stream (Content-Type: application/pdf).

Deliverables & Evaluation Criteria

1. **Working PDF Download:** Calling the PDF route delivers a clean, visually accurate PDF matching the target design.
2. **Template Architecture:** Clean separation between schema validation, data transformation, and template presentation.
3. **State Integrity:** Strict transition rules and error handling preventing invalid document states.
4. **Schema & Data Modeling:** Relational integrity between documents and audit logs.
5. **Containerization:** Functional Dockerfile or docker-compose.yml that builds and runs the service cleanly.

Submission Instructions

- Push your implementation to a GitHub repository.
- Include a README.md with:
 - Setup and run instructions (bun run dev or docker compose up).
 - The exact curl command or browser URL to trigger and verify the PDF download.
 - Architectural overview and design tradeoffs.
- **Deadline: EOD of Sept 22, 2026.**